

Computability: Recitation 8

May 18, 2026

1 Independent set and vertex cover

Definition 1 (Vertex Cover). Let $G = (V, E)$ be an undirected graph. A vertex cover is a set of vertices $C \subseteq V$ that touches every edge, i.e. for every $(u, v) \in E$ we have $u \in C$ or $v \in C$ (can be both). VC is the corresponding decision problem:

$$\text{VC} \stackrel{\text{def}}{=} \{\langle G, k \rangle : \text{There is a vertex cover of size } k \text{ in } G\}$$

Proposition 2. $\text{IS} \leq_p \text{VC}$.

Proof. The reduction is based on the following connection between vertex covers and independent sets: $S \subseteq V$ is independent iff $V \setminus S$ is a vertex cover. Indeed, if S is independent, then there are no edges inside S , so each edge touches at least one vertex in $V \setminus S$. Conversely, if $V \setminus S$ is a vertex cover, then every edge touches a vertex outside S , so there are no edges in S .

- **Construction:** Let $f(\langle G, k \rangle) = \langle G, |V| - k \rangle$.

Note: if $k > |V|$, the input is not in IS so we map it to some word outside VC, such as ϵ . Like invalid encodings, this is only a small remark and not central to the proof.

- **Correctness:** Suppose G contains an independent set $S \subseteq V$ of size k . By the observation above, $V \setminus S$ is a vertex cover, and its size is $|V| - |S| = |V| - k$. Conversely, suppose G contains a vertex cover C of size $|V| - k$. Then by the observation above, $V \setminus C$ is an independent set, and its size is $|V| - (|V| - k) = k$.
- **Complexity:** To implement the reduction we count the number of vertices $|V|$ and subtract k . We have seen that such arithmetic operations can be done in polynomial time.

□

A language L is called NP-hard if there is a polynomial reduction from every language in NP to L . If L is also in NP, it is called NP-complete. The reductions we have shown so far:

$$3\text{-SAT} \leq_p \text{CLIQUE} \leq_p \text{IS} \leq_p \text{VC}$$

These problems are in NP: a non-deterministic machine can guess a satisfying assignment (or a clique, or a cover, etc.) and check whether it works in polynomial time. These problems are also NP-hard: next week we will see that every $L \in \text{NP}$ can be reduced to $k\text{-SAT}$, which in turn can be reduced to 3-SAT.

2 Hamiltonian path and 3-SAT

Definition 3. A CNF (conjunctive normal form) boolean formula is "ANDs of ORs". That is, a conjunction (AND) of one or more clauses where each clause is a disjunction (OR) of literals. A literal is a variable or its negation.

Example: $(x_1) \wedge (\overline{x_2} \vee x_3)$ is a CNF formula. $(x_1 \wedge x_2) \vee (x_3)$ is not CNF.

Definition 4. A boolean formula is called satisfiable if there is an assignment to the variables such that the formula evaluates to TRUE.

Example : The formula $x \wedge \bar{x}$ is not satisfiable; $x_1 \vee x_2$ is satisfiable. For example, a satisfying assignment is $x_1 = \text{TRUE}, x_2 = \text{FALSE}$.

Definition 5. A formula is called 3-CNF if it is CNF and each clause has exactly 3 literals. The corresponding decision problem of satisfiability is:

$$3\text{-SAT} \stackrel{\text{def}}{=} \{\langle \varphi \rangle : \varphi \text{ is a satisfiable 3-CNF formula}\}$$

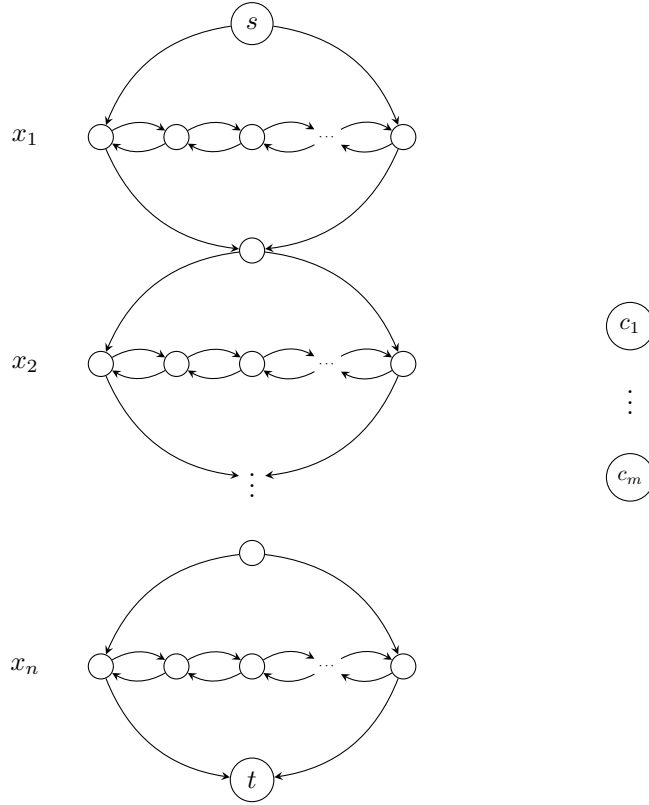
Definition 6. Let $G = (V, E)$ be a graph. A Hamiltonian path in G is a path that visits every vertex in G exactly once. D-ST-HAMPATH is the corresponding decision problem:

$$\text{D-ST-HAMPATH} \stackrel{\text{def}}{=} \{\langle G, s, t \rangle : G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t\}$$

Proposition 7. We have $3\text{-SAT} \leq_p \text{D-ST-HAMPATH}$.

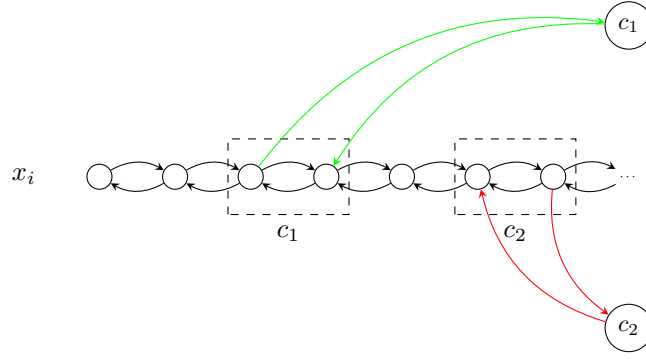
Proof. Suppose that φ is a 3-CNF formula with n variables $x_1, \dots, x_n$ and m clauses $c_1, \dots, c_m$. We construct a graph as in the following figure. Each variable is associated with a “diamond gadget”, and in it a horizontal chain of vertices that can be traversed left-to-right or right-to-left.

A path from s to t that visits once every vertex in the gadgets must traverse each chain in one of the possible directions. We arbitrarily associate left-to-right with TRUE and right-to-left with FALSE. Thus, each such path corresponds to a unique assignment to the variables, and vice versa: given a direction for each chain, we can construct a unique path accordingly.



In addition to the gadgets, we have a vertex for every clause. The idea is that visiting those vertices should correspond to satisfying the clauses, so that a Hamiltonian path from s to t exists iff a satisfying assignment exists. We do this by allowing the path to temporarily exit the chain, visit a clause that is satisfied by the variable's value in this assignment, and return to the chain. In every chain there are designated exit and return vertices for each clause.

For example, suppose that the literal x_i appears in c_1 , and the literal $\bar{x}_i$ appears in c_2 . Then the chain of x_i has the following extra edges:

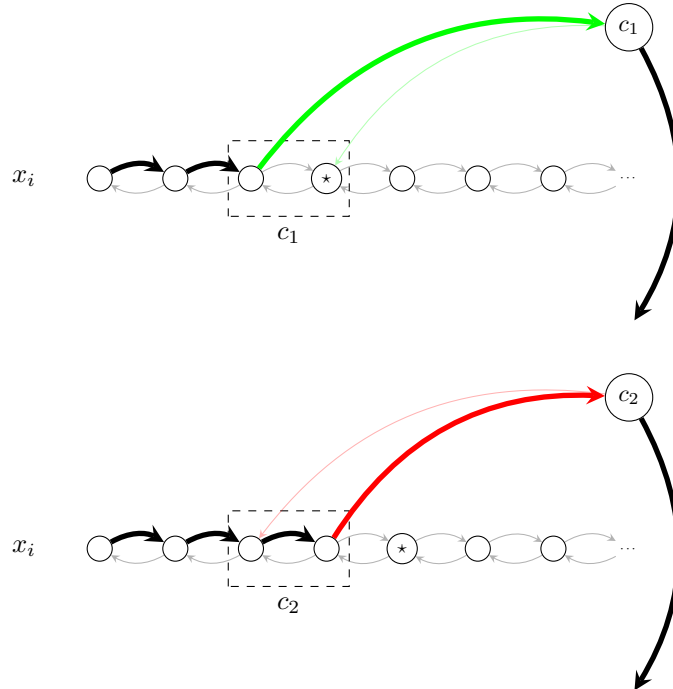


If a path traverses left-to-right, this corresponds to $x_i = \text{TRUE}$, so it has the option to visit c_1 and then go back to the chain and continue left-to-right (the directions of the colored arrows are important). Similarly, traversing right-to-left corresponds to $x_i = \text{FALSE}$ and such a path can detour to c_2 on its way. The path does not have to take these detours: we only visit the clause once throughout, from some literal that satisfies it. We note that there are separator vertices between pairs in the chain, which are used below.

Satisfying assignment $\rightarrow$ Hamiltonian path: Given a satisfying assignment, construct a path from s to t that traverses each chain in the direction corresponding to the variable's value. For each clause, choose a true literal (if there is more than one, we can arbitrarily choose the first) and add the corresponding detour to visit it. Such a literal exists because the assignment satisfies φ .

Hamiltonian path $\rightarrow$ Satisfying assignment: Suppose that on every detour to a clause, the path returns to the chain in the way we described. Then we can construct an assignment based on the traversal direction of each chain. Each clause is visited from some chain, and is hence satisfied by that variable's value. It is left to show that a path cannot “cheat” by jumping to a place we did not intend.

Consider the first time such a jump happens. Schematically, there are two ways the path may look, highlighted in bold. In the first, the path takes a valid detour, but continues to a different chain. In the second, the path jumps to a clause that is not satisfied by its current value.



In both cases, the path cannot reach the vertex marked $\star$. In the first case, the only way to visit it is from the right, but then the path cannot continue (and it must end in t). The same is true in the second case, because that vertex is a separator.

To summarize:

- **Construction:** Let $f(\langle\varphi\rangle) = \langle G, s, t \rangle$, as described above.
- **Correctness:** In the above analysis we showed that φ has a satisfying assignment iff there is a Hamiltonian path from s to t in G .
- **Complexity:** There are n gadgets, which is polynomial. To construct one such gadget, we add $O(m)$ vertices to the chain: two for each clause, and separators. For each clause, we check whether the current variable appears in it, whether it is negated, and connect accordingly. The overall running time is polynomial.

□

3 CNF formulas

Definition 8. A boolean formula is called k -CNF if it is CNF and each clause has exactly k literals. The corresponding decision problem:

$$k\text{-SAT} = \{\langle\varphi\rangle : \varphi \text{ is a satisfiable } k\text{-CNF formula}\}$$

Proposition 9. We have $k\text{-SAT} \leq_p 3\text{-SAT}$.

Let $k > 3$ (the case $k \leq 3$ is left as an exercise). We first show how to shorten a clause by one. Let $x_1, \dots, x_n$ be variables, and let $\phi = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$ be a clause of k literals. The idea is to introduce a new variable y that acts as $\ell_1 \vee \ell_2$. Consider the formula:

$$\phi' = (y \vee \ell_3 \vee \ell_4 \vee \dots \vee \ell_k) \wedge (\overline{\ell_1} \vee y \vee y) \wedge (\overline{\ell_2} \vee y \vee y) \wedge (\ell_1 \vee \ell_2 \vee \overline{y})$$

It is left as an exercise to check that the three clauses on the right are logically equivalent to $y \Leftrightarrow (\ell_1 \vee \ell_2)$.

Lemma 10. Every assignment $A : \{x_1, \dots, x_n\} \rightarrow \{\text{TRUE}, \text{FALSE}\}$ that satisfies ψ can be extended to an assignment $A' : \{x_1, \dots, x_n, y\} \rightarrow \{\text{TRUE}, \text{FALSE}\}$ that satisfies ψ' . In the other direction, every assignment $A' : \{x_1, \dots, x_n, y\} \rightarrow \{\text{TRUE}, \text{FALSE}\}$ that satisfies ψ' also satisfies ψ when restricted to $\{x_1, \dots, x_n\}$.

Proof. Let $A : \{x_1, \dots, x_n\} \rightarrow \{\text{TRUE}, \text{FALSE}\}$ be an assignment that satisfies ψ . Let A' be its extension to $\{x_1, \dots, x_n, y\}$ where $A'(y)$ is defined by the truth value that A gives to $\ell_1 \vee \ell_2$. Since the three new clauses are equivalent to $y \Leftrightarrow (\ell_1 \vee \ell_2)$, A' satisfies them. Since A' agrees with A , it satisfies $(\ell_1 \vee \dots \vee \ell_k)$ and hence ψ' .

In the other direction, let $A' : \{x_1, \dots, x_n, y\} \rightarrow \{\text{TRUE}, \text{FALSE}\}$ be an assignment that satisfies ψ' . Then it must give the same value to y and to $\ell_1 \vee \ell_2$. Since $(y \vee \ell_3 \vee \dots \vee \ell_k)$ is satisfied, its restriction to $\{x_1, \dots, x_n\}$ satisfies $(\ell_1 \vee \dots \vee \ell_k)$. □

Proof of the reduction.

- **Construction:** Let $f(\langle\varphi\rangle) = \langle\psi\rangle$, where ψ is 3-CNF and obtained from φ by repeatedly applying the above procedure to each clause. On each application, a new variable is introduced, and the clause size decreases (until it has size 3).
- **Correctness:** If $\langle\varphi\rangle \in k\text{-SAT}$, then there is a satisfying assignment to φ . By the lemma, we can extend it to a satisfying assignment to ψ by choosing the new variables accordingly (such as $y \Leftrightarrow (\ell_1 \vee \ell_2)$ in the example). Hence $\langle\psi\rangle \in 3\text{-SAT}$.
If $\langle\psi\rangle \in 3\text{-SAT}$, then there is a satisfying assignment to ψ . By the lemma, if we restrict the assignment to the variables in φ , it satisfies φ . Hence $\langle\varphi\rangle \in k\text{-SAT}$.
- **Complexity:** The procedure of adding a variable takes polynomial time: we search for a clause longer than 3, make a new variable name for it, and add the new clauses as above. The number of clauses in φ is bounded above by the input size, and so is the number of times the procedure must run per clause (it is bounded above by k). Therefore f can be computed in polynomial time.

□

4 Two Tapes

4.1 Motivation

Recall that RE is closed under union. For TMs M_1, M_2 that recognize L_1, L_2 respectively, we constructed a machine M that recognizes $L_1 \cup L_2$ by running M_1 and M_2 “in parallel.”

This task, like many others we will see, becomes easier if we allow a TM to have two tapes and two heads that it can use in every step. We can define M so that it copies the input w to the second tape, and then runs M_1 and M_2 simultaneously. Every pair of states from M_1 and M_2 corresponds to a single state in M , and the transition function advances both components separately, much like a product automaton.

4.2 Formal Definition (if time permits)

Recall that a single-tape TM transition function is

$$\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}.$$

For two tapes, δ must read one symbol from each tape, output two symbols, and move left or right on each tape:

$$\delta : (Q \setminus F) \times \Gamma^2 \rightarrow Q \times \Gamma^2 \times \{L, R\}^2.$$

For example, $\delta(q, \gamma_1, \gamma_2) = (q', \gamma'_1, \gamma'_2, L, R)$ means: when in state q , with head 1 reading γ_1 and head 2 reading γ_2 , the machine moves to q' , writes γ'_1 on tape 1 and γ'_2 on tape 2, and moves left on tape 1 and right on tape 2.

4.3 Simulating Two Tapes with One Tape

Theorem 11. *For every two-tape TM M , there exists an equivalent single-tape TM M' .*

Proof. There are several ways to prove this theorem. The general idea is to simulate the actions of the two tapes using just one tape.

A simple approach is to write the two tapes consecutively on one tape, separated by a special character $\#$. This construction works, but here we present a different solution.

Let $M = \langle \Sigma, \Gamma, \sqcup, Q, q_0, F, \delta \rangle$ be the two-tape TM. We construct

$$M' = \langle \Sigma, \Gamma', \sqcup, Q', q'_0, F, \delta' \rangle$$

as follows.

- The tape alphabet Γ' simulates both tapes and the positions of the two heads:

$$\Gamma' = (\Gamma \times \Gamma \times \{0, 1\} \times \{0, 1\}) \cup \Sigma \cup \{\sqcup\}.$$

Here, a symbol (a, b, h_1, h_2) means that this cell holds symbol a from tape 1 and symbol b from tape 2, with head i at this position if $h_i = 1$.

- M' starts by replacing each $\sigma \in \Sigma$ with $(\sigma, \sqcup, 0, 0)$. It then marks the first cell as $(\sigma, \sqcup, 1, 1)$, indicating both heads start at the leftmost cell.
- The simulation phase: M' encodes the state of M in its own state. For each simulated step:
 1. It scans the tape left-to-right to locate head 1, storing the scanned symbol in its state.
 2. It continues scanning to locate head 2 and stores that symbol.
 3. Now M' has q, γ_1, γ_2 and can apply δ .
 4. It scans again, updating the symbol under head 1 and moving the head (by changing h_1 bits appropriately), then repeats for head 2.

Note that Q' and Γ' are larger than Q and Γ but still finite.

Remark. Think of M' as an *emulator* for M . It encodes M 's configuration and simulates each of its steps by scanning and updating. This illustrates the general principle of simulating more powerful machines with simpler ones at a cost of efficiency. $\square$

4.4 Time Complexity of the Simulation

When translating between models, we care about the *cost* of simulation. For DFA and NFA, this cost was a blowup in states or transitions. For TMs, two key considerations arise:

- The *size* of the new machine: number of states, transitions, alphabet size.
- The *runtime* of the new machine on an input.

The size is straightforward but less interesting. Runtime is crucial: TMs can take very long to halt (if at all).

In the construction above, each simulated step of M requires $O(n)$ scanning steps on the tape, where n is the distance between the farthest head positions. After t steps of M on input w , the heads can be at most t cells apart. Thus, M' 's runtime is

$$O(t \cdot t) = O(t^2),$$

4.5 Generalizing to k Tapes

Proposition 12. *For every k -tapes TM M , there exists an equivalent single-tape TM M' .*

Furthermore, if M runs in $O(f(n))$, then M' runs in $O(f(n)^2)$.

Sketch. The construction generalizes naturally. Now, each tape cell encodes k symbols and k head positions:

$$\Gamma' = (\Gamma^k \times \{0, 1\}^k) \cup \Sigma \cup \{\sqcup\}.$$

Each step of M requires scanning the tape $O(f(n))$ times for each head, for a total of $O(k \cdot f(n) \cdot f(n)) = O(f(n)^2)$ runtime. $\square$

Corollary 13. *Every k -tapes TM M that runs in polynomial time, is equivalent to a single-tape TM M' that runs in polynomial time.*